



Higher National Diploma in Computing

(Software Engineering)

Unit 1: Programming

Evaluation Report

North Sussex Judo – Monthly Training Fee Calculator

Academic Year: 2024/2025

Assignment Title: Introduction to Algorithms and Programming with an IDE

Name: Vihanga Shamod Hewanayake

Student ID: 521546

1. Introduction

This report is a critical, reflective assessment of the project North Sussex Judo Monthly Training Fee Calculator. I will evaluate the accuracy and effectiveness of the algorithm in the final Python program, discuss the contributions of the Integrated Development Environment (IDE) to the development process and discuss what it would have looked like without the use of an IDE, and discuss the importance of coding standards for the individual developer and for the team of developers (AQ Digital Solutions (AQDS) in this case. In this report I have tried to be genuine about the positive elements of the project, as well as the areas for improvement, without being “politically correct”.

2. Evaluating the Algorithm Implementation

2.1 How well the Algorithm was Implemented.

I have commented my pseudocode design at the beginning of this project and compared it with the final 296 line main.py and am happy that the program has been implemented in the main.py program in all its main features. Each branch of the pseudocode is implemented by a function in the code. All fee formulae are reproduced verbatim. The loop UNTIL VALID pseudocode steps were implemented as the while True loops found in the ask_string(), ask_number(), ask_yes_no(), choose_plan() and choose_category() functions. The four menu branches were converted to four handler functions – handle_register(), handle_report(), handle_summary() and handle_exit() – which were all stored in a dictionary dispatch table.

The biggest feature added in the code that wasn't in the original pseudocode was the dispatch table, which replaced an IF/ELIF structure that used to chain together through the pseudocode. In pseudocode I took the approach of having one IF/ELIF loop, since conditionally routing in plain English is most naturally done using a series of sequential IF/ELIF branches. I was writing the Python code and realised that it was easier, more extensible and more Pythonic to use a dictionary of string keys to lambda functions. This was not designed into the algorithm, but rather occurred as a result of coding. This is a good example of a translation from algorithm to code that is not necessarily one way; coding can inform the design as well.

2.2 The Relationship Between Written Algorithm and Code

This project is unusual in that the pseudocode and the final code are very closely related. Nearly all pseudocode blocks are directly associated to a labelled section of a function or a named function. A reader who has only seen the pseudocode would be able to read through the code of main.py and follow it with little difficulty. This is the degree of quality of the initial algorithm design: if the algorithm is well-structured then the code flows naturally.

The biggest difference between the algorithm and the code is input validation. Pseudocode mimics the appearance of a clean and valid string coming in. There are various conditions that need to be satisfied in real Python code: blank strings, failure to convert, out-of-range values, and duplicate names. The five validation functions: ask_string(), ask_number(), ask_yes_no(), choose_plan(), choose_category() are the middle ground between the simplified algorithm and the grittier real-life aspect of user input. If I were writing the pseudocode again, I would explicitly have VALIDATE steps after each READ, as this would at least be a substantial portion of the code left out of the pseudocode and could lead to misunderstandings by those who use the pseudocode as a specification.

2.3 Critical Assessment: What Would I Do Differently?

The pseudocode could be enhanced by a `DEFINE ATHLETE` section prior to the main loop to explicitly say what the Athlete object contains, as this would be useful when considering the algorithm critically. The Athlete is first introduced in the current pseudocode in the register branch, but no description of what attributes the Athlete carries. This seemed logical to me because already knew I would be writing a class, but if you read the pseudocode cold you wouldn't know that. If the algorithm were to be made explicit in the data model, then the pseudocode would provide a more true-to-life specification of the algorithm than a personal design aid.

I wouldn't make the dispatch table itself simple, simple `IF/ELIF` either; it's a design choice with real consequences as to how the program is extended. Previously, pseudocode had been used that already represented the final design of the architecture, minimizing the gap between design and implementation.

3. Evaluating the Use of the IDE

3.1 The IDE used and What it Provided?

This project was done using Visual Studio Code (VS Code) and the Microsoft Python extension. For syntax highlighting, IntelliSense auto-completion, step and inspect in an integrated debugger, built-in terminal, and native Git integration, VS Code provided all the features. None of these features were simply offered, rather they were used during development.

3.2 Concrete Contribution of the IDE

The built-in debugger is the most useful feature that I used. At the start of testing the Heavyweight weight class crashed with a `TypeError`; so I set a breakpoint in `weight_status()`, ran the program in debug mode and registered a Heavyweight athlete to cause the crash. VS Code has stopped the program at a breakpoint and showed what the value of each variable that is in scope is. It was clear from the variable panel that the subtraction was trying to happen with `None` as `cat_limit`. I added a `None` guard at the beginning of `weight_status()` before any arithmetic, to correct the algorithm. This was completed in less than 10 minutes. The same fix, without the debugger, would take: reading the error message, speculating which variable is `None`, adding a print statement, running the program again, crashing, reading the print statement, removing the print statement, writing the fix, running the program again. This focussed debugging session collapsed the above.

Throughout the development, but not as much as it was in the initial stages, IntelliSense proved helpful. VS Code showed the method signature and docstring in a tooltip when calling the methods on the Athlete object, even though I did not scroll back to the definition of the Athlete class. It also caught out a typo in a variable name before I ran the program, which would have resulted in a `NameError` when it was run. The Git integration panel offered visual diff before each commit, which helped me to write more accurate commit messages and make sure that the six commit history is faithful to the project's construction.

124d0f3	Added Event handlers and main loop
d7b0bea	Added Display functions
3a64eca	Added input validations
c9fec1f	Added calculation functions

75eb17f	Added constants and Athlete classes
3689caf	Initial commit-README

3.3 What Development Would Have Looked Like Without an IDE

If it were not for VS Code, I would have written Python in a plain text editor and ran it in the command line. That is indeed possible; Python doesn't need an IDE, just the interpreter... but there would have been various concrete differences I could point to that would not have been pleasant to develop on.

If we hadn't had the integrated debugger we would have had to do print-statement debugging for everything. This would have taken several edit read runs for the Heavyweight crash. If the logic error is more subtle, like incorrectly applying some kind of coaching hours cap, then print debugging is downright difficult—the answers to the questions of which variables to print at what point can only be guessed beforehand without knowing the issue. The VS Code debugger eliminates this ambiguity altogether, and displays all of the variable values at point of pause.

If IntelliSense wasn't available, each method name and attribute would have to be discovered by memory or by scrolling back to the class definition. This is feasible in a 296-line project or one that runs in a single file. For more complex, multi-file assemblies, or for any file-based project that has been away for several weeks, the mental burden grows and typing errors commonly occur.

But if I hadn't used the integrated Git panel I would still have been using version control, but I would be switching back and forth between the editor and the terminal every single commit; that's umpteen more steps and I'm pretty sure that meant I'd have been less precise in putting in my messages, and I'd certainly have been less frequent, in committing. An obvious chronology of the commits, which can be seen from the 6-commits History, is likely what the commit would have been (two or three of them a bit bigger in size and would be less readable; hard to follow the development sequence).

To summarize: the program could not have ever been written without the IDE, but the IDE made it faster, easier to program correctly and better documented. Each of those difference is of real value in a company that provides software to real clients on real commercial deadlines, such as AQDS.

4. Evaluating the Role and Purpose of Coding Standards.

4.1 What Coding Standards are and what they do?

A coding standard is a standard of conventions (naming, formatting, documentation structure, file organisation etc.) that a team agrees upon to be followed by each member of the team, regardless of their own preference. This project implements PEP 8 ("The Python Style Guide") (2023) — the official Python style guide maintained by the Python Software Foundation. PEP 8 is a style guide for the "naming rules", the "indentation rules", line length, blank lines, comment style and import ordering.

Enforcing a coding standard throughout a code base is much more powerful than the sum of its individual rules. Namings conventions that are defined by the syntax (verb_noun for Functions, UPPER_CASE for Constants, ClassName for Classes) make the code structurally predictable. Extending this class is straightforward, and a developer who has never seen main.py can immediately locate the calculation logic in the calc_* functions, the validation logic in the ask_* and choose_* functions, the display logic in the print_* functions, and the event handling logic in the handle_* functions. This navigability does not happen accidentally; it is the direct product of applied standards.

4.2 Why Coding Standards Matter for Individual Developers.

Even without team members, PEP 8 still provided tangible benefits from this project. After some days, when I went back in to mock up the defensive `min()` cap, the function's name, the descriptive comments in the comments section, and the short, descriptive variable names (from hours to WEEKS, and from COACHING_RATE to MAX_COACHING) enabled me to understand and make the change in seconds and with confidence. If I had called the function `f2()` and also given the variables variables such as `c`, `h` and `r`, I would have to repeat scanning the entire function before figuring it out. Self-review is also implemented more systematically and effectively through the standards. I checked changed code all against the PEP 8 checklist for each Git commit. This process picked up a partial lack of a class docstring, which took a small step more than 79 characters, and inconsistency in quoting style. These kind of problems would not have been runtime problems but more of a lack of professionalism and lack of uniformity in the code. Before committing them, the version history will include impeccable code that is always formatted consistently.

4.3 Why Coding Standards Are Essential in a Team

The significance of coding standards is significantly enhanced in a professional team setting. I would like my code reviewed at AQDS by other more knowledgeable coders; and perhaps be maintained once developed by other colleagues not involved in the project. There is no consensus, each developer's code is unique, containing different code formatting of naming, indentation, documentation, function structure etc. The logic has to be understood and then subsequently reviewed, and by the time a reviewer gets a chance to do this, every individual developer has taken some time to get used to each other's coding style. Because developers make structural assumptions about the code base that may not be warranted, with the exception of maintenance. The exception of maintenance, because with it, code becomes riskier, because the developer acts in accordance with the various assumptions for its structure.

In an empirical study of two software development teams, Popic (2018) has observed that enforcing a common rule increases the efficiency of code review in terms of number of defects found, review time and the developer's confidence after modifying code that they were not responsible for writing. This is what really counts in a commercial software development company. A client such as North Sussex Judo want software which operates reliably, works correctly and can be maintained and extended as the club's needs change. One of the core aspects of all three of those things is shared coding standards. Standards also fast track onboarding. Having everything in the codebase conform to PEP 8 means that when a new developer joins AQDS to open an existing project, they can meaningfully contribute much sooner, since the structure is the same whenever the domain logic is new. The time saved in a whole organization throughout the year of training and paying for new employees is not small!

4.4 An Honest Reflection on Limitations

I'm on the honest side and will say that following PEP 8 did not eliminate all the issues in this project! The Heavyweight None crash didn't get caught by any sense of style guide. The original `if/elif` dispatch chain was a "structural" vs. a style one. Coding standards are not enough for quality software. They should go hand in hand with careful design of algorithms, comprehensive testing and debugging with the IDE. Standards provide a professional baseline for code quality – anything above the standard will require skill and experience, along with good engineering judgement.

5. Conclusion

As a part of this project overall, I believe that the algorithm was correctly and effectively implemented in the end program, that the IDE had a definite measurable impact on the quality and efficiency of programming, that the coding standards I used make the code base professionally maintainable to any AQDS programmer. The single most important takeaway from this project for me is that these three aspects - algorithm design, tooling for the IDE, coding standards - do not work independently and each does not provide the value it can when it operates within the others. Without writing one line of code, the algorithm did provide me with a clear, correct strategy validated. With IDE, I could run this plan in an effective manner and correct errors accurately and rapidly. The coding standards make sure that it's professional, consistent, and is easy to navigate. Together, they form a practice development that I plan to apply to all projects that I become a part of at AQDS, and one that I would push for in future teams.

References

Aho, A.V., Hopcroft, J.E. and Ullman, J.D. (1987) Data Structures and Algorithms. 1st edn. Reading, MA: Addison-Wesley.

Adragna, P. (n.d.) Software Debugging Techniques. CERN. Available at: <https://cds.cern.ch/record/1100526/files/p71.pdf> (Accessed: 10 June 2024).

Edwards, S., Shaffer, C.A. and Kapur, A. (2012) Problem Solving and Algorithms. Available at: <http://sofia.cs.vt.edu/cs1114-ebooklet/chapter4.html> (Accessed: 10 June 2024).

McConnell, S. (2004) Code Complete: A Practical Handbook of Software Construction. 2nd edn. Redmond, WA: Microsoft Press.

McCauley, R. et al. (2008) 'Debugging: A Review of the Literature from an Educational Perspective', Computer Science Education, 18(2), pp. 67–92. doi:10.1080/08993400802114581.

Mitchell, J. (2002) Concepts in Programming Languages. Cambridge: Cambridge University Press.

Popic, S. (2018) 'The Benefits of the Coding Standards Enforcement and its Influence on the Developers' Coding Behaviour: A Case Study on Two Small Projects', 26th Telecommunications Forum TELFOR 2018. doi:10.1109/TELFOR.2018.8612149.

Python Software Foundation (2023) PEP 8 – Style Guide for Python Code. Available at: <https://peps.python.org/pep-0008/> (Accessed: 10 June 2024).

Reeves, J. (1992) 'What is Software Design?', C++ Journal. Available at: <http://www-public.telecom-sudparis.eu/~gibson/Teaching/Teaching-ReadingMaterial/Reeves92.pdf> (Accessed: 10 June 2024).